

LangChain & Agentic AI – From Basics to Advanced demo-11 Notes

What is ChatPromptTemplate:

It is used to create dynamic reusable and message based prompts.

Note:

PromptTemplate --->Text based dynamic prompts

ChatPromptTemplate --->Message based dynamic prompts

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import SystemMessage,  
HumanMessage, AIMessage
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
template = ChatPromptTemplate.from_messages(
    [
        SystemMessage("You are a helpful Python Teacher"),
        HumanMessage("Explain Loops"),
        AIMessage("Loops are important concepts in Python"),
        HumanMessage("Explain Conditional Statements")
    ]
)

#print(type(template))

#print(template)

prompt = template.invoke({})

print(type(prompt))

for message in prompt.messages:
    print(type(message).__name__, ":", message.content)
```

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

response = llm.invoke(prompt) #ChatPromptValue is the child
class of PromptValue
print(response.content)
```

=====

```
import os

from langchain_openai import ChatOpenAI

from langchain_core.messages import SystemMessage,
HumanMessage, AIMessage

from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages(
    [
        ('system', "You are a helpful Python Teacher"),
        ('human', "Explain Loops"),
```

```
(('ai', "Loops are important concepts in Python"),
 ('human',
  "Explain switch Statement"
 )
]
)

#print(type(template))
#print(template)

prompt = template.invoke({})
print(type(prompt))

for message in prompt.messages:
    print(type(message).__name__, ":", message.content)

llm = ChatOpenAI(
    model="gpt-4o-mini",
```

```

    api_key=os.getenv("OPENAI_API_KEY")
)
response = llm.invoke(prompt) #ChatPromptValue is the child
class of PromptValue
print(response.content)

=====

import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages(
    [
        ('system', "You are a helpful Python Teacher, Explain
Everything in simple english"),
        ('human', "Explain {topic} with an example")
    ]
)

#print(template.input_variables)

prompt = template.invoke( {"topic":"Decorators"})

```

```
for message in prompt.messages:
    print(type(message).__name__,":",message.content)
```

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

response = llm.invoke(prompt) #ChatPromptValue is the child
class of PromptValue
print(response.content)
```

```
=====
```

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages(
```

```
[  
    ('system', "You are a helpful {subject} Teacher, Explain  
Everything in simple english"),  
    ('human', "Explain {topic} with an example")  
]  
)
```

```
subject = input("Enter Subject:")
```

```
topic = input("Enter Topic:")
```

```
prompt = template.invoke( {"subject":subject, "topic":topic})
```

```
for message in prompt.messages:
```

```
    print(type(message).__name__, ":", message.content)
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)  
  
response = llm.invoke(prompt) #ChatPromptValue is the child  
class of PromptValue  
  
print(response.content)
```

```
=====
```

```
import os  
  
from langchain_openai import ChatOpenAI  
from langchain_core.prompts import ChatPromptTemplate  
  
# Create chat model  
  
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)  
  
# Create dynamic chat prompt  
  
template = ChatPromptTemplate.from_messages(  
    [  

```

(

"system",

""

You are an expert {subject} teacher.

Student Level: {level}

Language: {language}

Your job is to teach concepts clearly
and use easy examples.

""

),

(

"human",

""

Teach me {topic}.

Requirements:

1. Give a simple definition.
2. Explain why it is required.
3. Give 5 important points.
4. Give 2 real-life examples.
5. Mention common mistakes.
6. End with a one-line conclusion.

""""

)

]

)

Get values from student

subject = input("Enter subject: ")

topic = input("Enter topic: ")

level = input("Enter student level: ")

language = input("Enter language: ")

Format chat prompt

```
prompt = template.invoke(  
    {  
        "subject": subject,  
        "topic": topic,  
        "level": level,  
        "language": language  
    }  
)
```

```
# Display generated messages
```

```
print("\n===== GENERATED MESSAGES =====")
```

```
for message in prompt.messages:
```

```
    print("\nMessage Type:")
```

```
    print(type(message).__name__)
```

```
    print("Content:")
```

```
    print(message.content)
```

```
# Call model
```

```
response = llm.invoke(prompt)
```

```
# Display final response
```

```
print("\n===== AI RESPONSE =====")
```

```
print(response.content)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
# Create chat model
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
api_key=os.getenv("OPENAI_API_KEY")
)

template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are an expert in {topic}."
        ),
        (
            "human",
            "Explain {topic} to me."
        )
    ]
)

# Format chat prompt
prompt = template.invoke(
    {
```

```
        "topic": "python"
    }
)

# Display generated messages
print("\n===== GENERATED MESSAGES =====")

for message in prompt.messages:
    print("\nMessage Type:")
    print(type(message).__name__)

    print("Content:")
    print(message.content)


# Call model
response = llm.invoke(prompt)
```

```
# Display final response
```

```
print("\n===== AI RESPONSE =====")
```

```
print(response.content)
```

```
=====
```

Important Questions:

Question 1

What is ChatPromptTemplate?

Answer:

A reusable template for creating dynamic chat messages.

Question 2

Where do we import it from?

```
from langchain_core.prompts import ChatPromptTemplate
```

Question 3

Which method can create it from message definitions?

ChatPromptTemplate.from_messages()

Question 4

Can system messages contain variables?

Answer:

Yes.

Question 5

Can human messages contain variables?

Answer:

Yes.

Question 6

Can AI messages be included?

Answer:

Yes.

Question 7

Does `template.invoke()` generate an AI answer?

Answer:

No.

It formats the chat prompt.

Question 8

Which call actually invokes the model?

`llm.invoke(...)`

Most Important Points

1. `ChatPromptTemplate` is meant for dynamic reusable chat-oriented prompts.
2. It can contain system, human and AI message definitions.
3. Message contents can contain variables.
4. Variables can appear in any message template.
5. `from_messages()` creates the chat prompt template.

6. `template.invoke()` fills variables and creates formatted messages.
7. `template.invoke()` does not call the LLM.
8. `llm.invoke()` calls the model.
9. The model returns an `AIMessage`.
10. `response.content` gives the generated answer content.